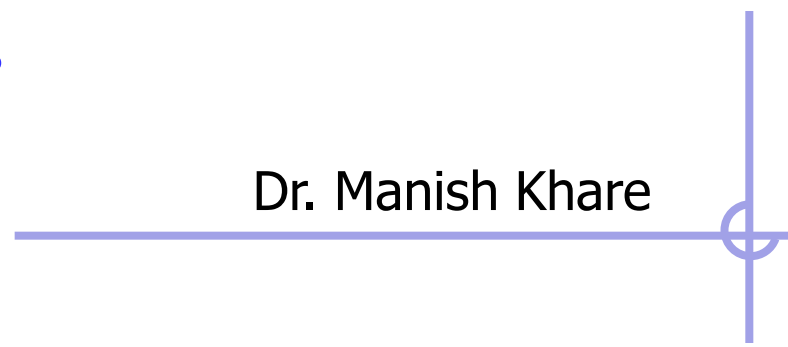


Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Hash Tables
Sets
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

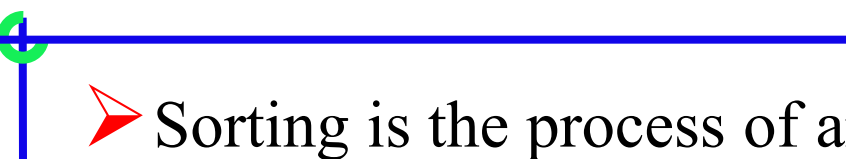




Sorting Algorithms

Sorting



- 
- Sorting is the process of arranging a list of elements in a particular order (Ascending or Descending).
 - Sorting is categorized as
 - Internal Sorting
 - External Sorting
 - Internal sorting means we are arranging the numbers within the array only which is in computer primary memory.
 - External sorting is the sorting of numbers from the external file by reading it from secondary memory.

- 
- Why do we need to sort data ?:
 - to arrange names in alphabetical order
 - arrange students by grade, etc.
 - preliminary step to searching data.
 - Basic steps involved:
 - compare two items
 - swap the two items or copy one item.



➤ Type of Sorting algorithm

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Radix Sort
- Merge Sort
- Heap Sort
- Shell Sort

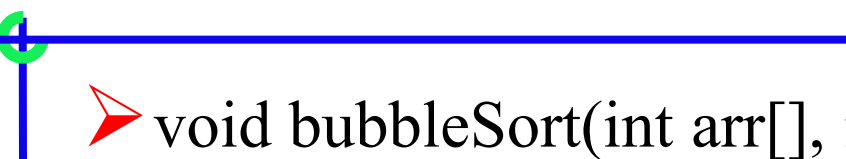
Bubble Sort

- Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.
- This algorithm is not suitable for large data sets

Bubble Sort

- In bubble sort, each element is compared with its adjacent element. If the first element is larger than the second one then the position of the elements are interchanged, otherwise it is not changed. Then the next element is compared with its adjacent element and the same process is repeated for all the elements in the array. At last largest element is at last position.
- During the next all passes the same process is repeated until no more elements are left for comparison.

Function for Bubble Sort



```
void bubbleSort(int arr[], int n)
```

```
{
```

```
    int i, j;
```

```
    for (i = 0; i < n-1; i++)
```

```
        // Last i elements are already in place
```

```
        for (j = 0; j < n-i-1; j++)
```

```
            if (arr[j] > arr[j+1])
```

```
                swap(&arr[j], &arr[j+1]);
```

```
}
```

Technique of Bubble Sort

- The basic methodology of the working of bubble sort is given as follows:
- In Pass 1, $A[0]$ and $A[1]$ are compared, then $A[1]$ is compared with $A[2]$, $A[2]$ is compared with $A[3]$, and so on. Finally, $A[N-2]$ is compared with $A[N-1]$. Pass 1 involves $n-1$ comparisons and places the biggest element at the highest index of the array.
- In Pass 2, $A[0]$ and $A[1]$ are compared, then $A[1]$ is compared with $A[2]$, $A[2]$ is compared with $A[3]$, and so on. Finally, $A[N-3]$ is compared with $A[N-2]$. Pass 2 involves $n-2$ comparisons and places the second biggest element at the second highest index of the array.
- In Pass 3, $A[0]$ and $A[1]$ are compared, then $A[1]$ is compared with $A[2]$, $A[2]$ is compared with $A[3]$, and so on. Finally, $A[N-4]$ is compared with $A[N-3]$. Pass 3 involves $n-3$ comparisons and places the third biggest element at the third highest index of the array.
- In Pass $n-1$, $A[0]$ and $A[1]$ are compared so that $A[0] < A[1]$. After this step, all the elements of the array are arranged in ascending order

Bubble Sort - Example 1

➤ **Example:** (5 1 4 2 8)

First Pass:

(5 1 4 2 8) $\rightarrow$ (1 5 4 2 8), Here, algorithm compares the first two elements, and swaps since $5 > 1$.

(1 5 4 2 8) $\rightarrow$ (1 4 5 2 8), Swap since $5 > 4$

(1 4 5 2 8) $\rightarrow$ (1 4 2 5 8), Swap since $5 > 2$

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8), Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

➤ **Second Pass:**

(1 4 2 5 8) $\rightarrow$ (1 4 2 5 8)

(1 4 2 5 8) $\rightarrow$ (1 2 4 5 8), Swap since $4 > 2$

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one **whole** pass without **any** swap to know it is sorted.

➤ **Third Pass:**

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

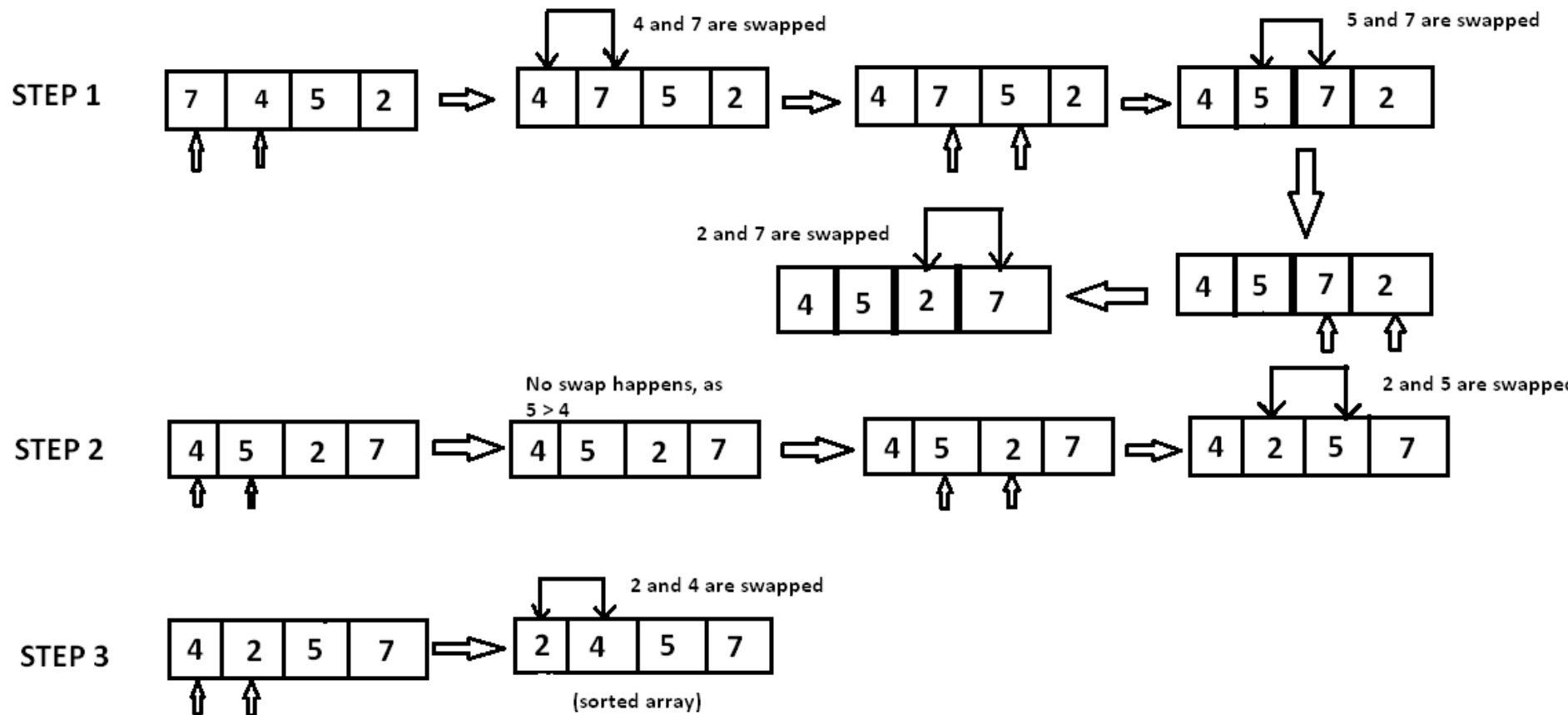
(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

(1 2 4 5 8) $\rightarrow$ (1 2 4 5 8)

Bubble Sort - Example 2

Example: { 7, 4, 5, 2 }



Bubble Sort - Exercise



Sort following array using Bubble Sort.

1. {54,26,93,17,77,31,44,55,20}

2. {56,47,41,35,22,11}

Complexity of Bubble Sort

- The complexity of any sorting algorithm depends upon the number of comparisons.
- In bubble sort, we have seen that there are $N-1$ passes in total. In the first pass, $N-1$ comparisons are made to place the highest element in its correct position. Then, in Pass 2, there are $N-2$ comparisons and the second highest element is placed in its position.
- Therefore, to compute the complexity of bubble sort, we need to calculate the total number of comparisons.


$$f(n) = (n - 1) + (n - 2) + (n - 3) + \dots + 3 + 2 + 1$$

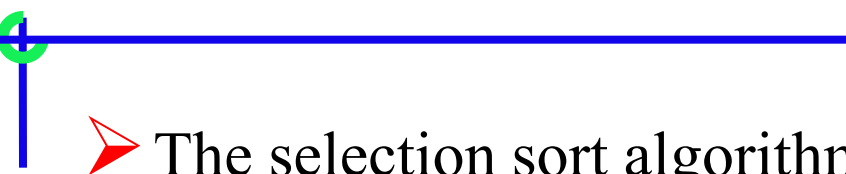
$$f(n) = n(n - 1)/2$$

$$f(n) = n^2/2 + O(n) = O(n^2)$$

- Therefore, the complexity of bubble sort algorithm is $O(n^2)$. It means the time required to execute bubble sort is proportional to n^2 , where n is the total number of elements in the array.

Selection Sort

- Selection Sort algorithm is used to arrange a list of elements in a particular order (Ascending or Descending).
- In selection sort, the first element in the list is selected and it is compared repeatedly with remaining all the elements in the list. If any element is smaller than the selected element (for Ascending order), then both are swapped.
- Then we select the element at second position in the list and it is compared with remaining all elements in the list. If any element is smaller than the selected element, then both are swapped. This procedure is repeated till the entire list is sorted.



➤ The selection sort algorithm is performed using following steps...

- **Step 1:** Select the first element of the list (i.e., Element at first position in the list).
- **Step 2:** Compare the selected element with all other elements in the list.
- **Step 3:** For every comparison, if any element is smaller than selected element (for Ascending order), then these two are swapped.
- **Step 4:** Repeat the same procedure with next position in the list till the entire list is sorted.

Function for Selection Sort

//Selection sort logic

```
for(i=0; i<size; i++){  
    for(j=i+1; j<size; j++){  
        if(list[i] > list[j])  
        {  
            temp=list[i];  
            list[i]=list[j];  
            list[j]=temp;  
        }  
    }  
}
```

Selection Sort – Example 1

Consider the following unsorted list of elements...

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

Iteration #1

Select the first position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

$15 > 20$
FALSE

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

$15 > 10$
TRUE
SWAP

Selection Sort – Example 1



$10 > 30$
FALSE



$10 > 50$
FALSE

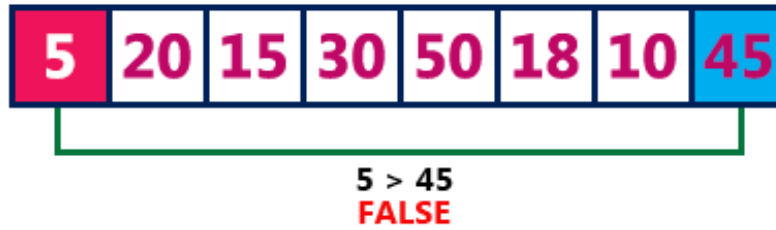


$10 > 18$
FALSE



$10 > 5$
TRUE
SWAP

Selection Sort – Example 1



List after 1st iteration



Iteration #2

Select the second position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 2nd iteration



Selection Sort – Example 1

Iteration #3

Select the third position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 3rd iteration

5	10	15	30	50	20	18	45
---	----	----	----	----	----	----	----

Iteration #4

Select the fourth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 4th iteration

5	10	15	18	50	30	20	45
---	----	----	----	----	----	----	----

Iteration #5

Select the fifth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 5th iteration

5	10	15	18	20	50	30	45
---	----	----	----	----	----	----	----

Selection Sort – Example 1

Iteration #6

Select the sixth position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 6th iteration



Iteration #7

Select the seventh position element in the list, compare it with all other elements in the list and whenever we found a smaller element than the element at first position then swap those two elements.

List after 7th iteration



Final sorted list

Technique of Selection Sort

- Consider an array ARR with N elements. Selection sort works as follows:
- First find the smallest value in the array and place it in the first position. Then, find the second smallest value in the array and place it in the second position. Repeat this procedure until the entire array is sorted.
- Therefore,
 - In Pass 1, find the position POS of the smallest value in the array and then swap $ARR[POS]$ and $ARR[0]$. Thus, $ARR[0]$ is sorted.
 - In Pass 2, find the position POS of the smallest value in sub-array of $N-1$ elements. Swap $ARR[POS]$ with $ARR[1]$. Now, $ARR[0]$ and $ARR[1]$ is sorted.
 - In Pass $N-1$, find the position POS of the smaller of the elements $ARR[N-2]$ and $ARR[N-1]$. Swap $ARR[POS]$ and $ARR[N-2]$
- so that $ARR[0], ARR[1], \dots, ARR[N-1]$ is sorted.

Selection Sort - Exercise

Sort following array using Selection Sort.

1. {16,15,2,13,6}

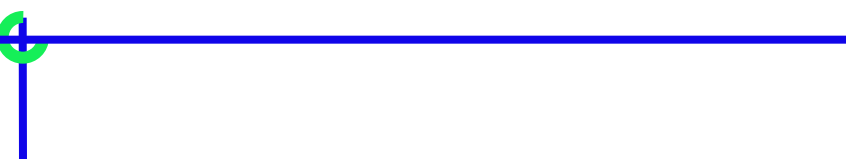
2. {44,77,22,88,33,66,11,55}

Complexity of Selection Sort

- Selection sort is a sorting algorithm that is independent of the original order of elements in the array.
- In Pass 1, selecting the element with the smallest value calls for scanning all n elements; thus, $n-1$ comparisons are required in the first pass. Then, the smallest value is swapped with the element in the first position.
- In Pass 2, selecting the second smallest value requires scanning the remaining $n - 1$ elements and so on.
- Therefore,
- $(n - 1) + (n - 2) + \dots + 2 + 1$
- $= n(n - 1) / 2 = O(n^2)$ comparisons

Insertion Sort

- Sorting is the process of arranging a list of elements in a particular order (Ascending or Descending).
- Insertion sort algorithm arranges a list of elements in a particular order. In insertion sort algorithm, every iteration moves an element from unsorted portion to sorted portion until all the elements are sorted in the list.



➤ The insertion sort algorithm is performed using following steps...

- **Step 1:** Assume that first element in the list is in sorted portion of the list and remaining all elements are in unsorted portion.
- **Step 2:** Consider first element from the unsorted list and insert that element into the sorted list in order specified.
- **Step 3:** Repeat the above process until all the elements from the unsorted list are moved into the sorted list.

Function for Insertion Sort

➤ //Insertion sort logic

```
for i = 1 to size-1 {  
    temp = list[i];  
    j = i;  
    while ((temp < list[j]) && (j > 0)) {  
        list[j] = list[j-1];  
        j = j - 1;  
    }  
    list[j] = temp;  
}
```

Technique of Insertion Sort

- Insertion sort works as follows:
- The array of values to be sorted is divided into two sets. One that stores sorted values and another that contains unsorted values.
- The sorting algorithm will proceed until there are no elements in the unsorted set.
- Suppose there are n elements in the array. Initially, the element with index 0 (assuming $LB = 0$) is in the sorted set. Rest of the elements are in the unsorted set.
- The first element of the unsorted partition has array index 1 (if $LB = 0$).
- During each iteration of the algorithm, the first element in the unsorted set is picked up and inserted into the correct position in the sorted set.

Insertion Sort – Example 1

Consider the following unsorted list of elements...

15	20	10	30	50	18	5	45
----	----	----	----	----	----	---	----

Assume that sorted portion of the list empty and all elements in the list are in unsorted portion of the list as shown in the figure below...

Sorted	Unsorted							
	15	20	10	30	50	18	5	45

Insertion Sort – Example 1

Move the first element 15 from unsorted portion to sorted portion of the list.



To move element 20 from unsorted to sorted portion, Compare 20 with 15 and insert it at correct position



Insertion Sort – Example 1

To move element 10 from unsorted to sorted portion, Compare 10 with 20 and it is smaller so swap. Then compare 10 with 15 again smaller swap. And 10 is insert at its correct position in sorted portion of the list.

Sorted			Unsorted				
10	15	20	30	50	18	5	45

To move element 30 from unsorted to sorted portion, Compare 30 with 20, 15 and 10. And it is larger than all these so 30 is directly inserted at last position in sorted portion of the list.

Sorted				Unsorted			
10	15	20	30	50	18	5	45

Insertion Sort – Example 1

To move element 50 from unsorted to sorted portion, Compare 50 with 30, 20, 15 and 10. And it is larger than all these so 50 is directly inserted at last position in sorted portion of the list.

Sorted					Unsorted		
10	15	20	30	50	18	5	45

To move element 18 from unsorted to sorted portion, Compare 18 with 30, 20 and 15. Since 18 is larger than 15, move 20, 30 and 50 one position to the right in the list and insert 18 after 15 in the sorted portion.

Sorted						Unsorted	
10	15	18	20	30	50	5	45

Insertion Sort – Example 1

To move element 5 from unsorted to sorted portion, Compare 5 with 50, 30, 20, 18, 15 and 10. Since 5 is smaller than all these element, move 10, 15, 18, 20, 30 and 50 one position to the right in the list and insert 5 at first position in the sorted list.

Sorted							Unsorted
5	10	15	18	20	30	50	45

To move element 45 from unsorted to sorted portion, Compare 45 with 50 and 30. Since 45 is larger than 30, move 50 one position to the right in the list and insert 45 after 30 in the sorted list.

Sorted							Unsorted
5	10	15	18	20	30	45	50

Insertion Sort – Example 1

Unsorted portion of the list has become empty. So we stop the process. And the final sorted list of elements is as follows...

5	10	15	18	20	30	45	50
---	----	----	----	----	----	----	----

Insertion Sort – Example 2

54	26	93	17	77	31	44	55	20	Assume 54 is a sorted list of 1 item
26	54	93	17	77	31	44	55	20	inserted 26
26	54	93	17	77	31	44	55	20	inserted 93
17	26	54	93	77	31	44	55	20	inserted 17
17	26	54	77	93	31	44	55	20	inserted 77
17	26	31	54	77	93	44	55	20	inserted 31
17	26	31	44	54	77	93	55	20	inserted 44
17	26	31	44	54	55	77	93	20	inserted 55
17	20	26	31	44	54	55	77	93	inserted 20

Insertion Sort – Exercise

Sort following array using Insertion Sort.

1. {25,15,30,9,99,20,26}
2. {44,77,22,88,33,66,11,55}

Complexity of Insertion Sort

- For insertion sort, the best case occurs when the array is already sorted. In this case, the running time of the algorithm has a linear running time (i.e., $O(n)$).
- This is because, during each iteration, the first element from the unsorted set is compared only with the last element of the sorted set of the array.
- Worst Case complexity is $O(n^2)$
- In the average case, the insertion sort algorithm will have to make at least $(K-1)/2$ comparisons. So the average case complexity is $O(n)$

Quick Sort

- Quick sort is a fast sorting algorithm used to sort a list of elements. Quick sort algorithm is invented by C. A. R. Hoare.
- The quick sort algorithm attempts to separate the list of elements into two parts and then sort each part recursively. That means it use divide and conquer strategy.
- In quick sort, the partition of the list is performed based on the element called pivot. Here pivot element is one of the elements in the list.

- 
- The list is divided into two partitions such that "all elements to the left of pivot are smaller than the pivot and all elements to the right of pivot are greater than or equal to the pivot".



Step by Step Process

- In Quick sort algorithm, partitioning of the list is performed using following steps...
- **Step 1** - Consider the first element of the list as **pivot** (i.e., Element at first position in the list).
- **Step 2** - Define two variables i and j . Set i and j to first and last elements of the list respectively.
- **Step 3** - Increment i until $\text{list}[i] > \text{pivot}$ then stop.
- **Step 4** - Decrement j until $\text{list}[j] < \text{pivot}$ then stop.
- **Step 5** - If $i < j$ then exchange $\text{list}[i]$ and $\text{list}[j]$.
- **Step 6** - Repeat steps 3,4 & 5 until $i > j$.
- **Step 7** - Exchange the pivot element with $\text{list}[j]$ element.

Technique of Quick Sort

Quick sort works as follows:

- 1. Set the index of the first element in the array to `loc` and `left` variables. Also, set the index of the last element of the array to the `right` variable.
 - That is, `loc = 0`, `left = 0`, and `right = n-1` (where `n` is the number of elements in the array)
- 2. Start from the element pointed by `right` and scan the array from right to left, comparing each element on the way with the element pointed by the variable `loc`.
 - That is, `a[loc]` should be less than `a[right]`.
 - (a) If that is the case, then simply continue comparing until `right` becomes equal to `loc`. Once `right = loc`, it means the pivot has been placed in its correct position.
 - (b) However, if at any point, we have `a[loc] > a[right]`, then interchange the two values and jump to Step 3.
 - (c) Set `loc = right`
- 3. Start from the element pointed by `left` and scan the array from left to right, comparing each element on the way with the element pointed by `loc`.
 - That is, `a[loc]` should be greater than `a[left]`.
 - (a) If that is the case, then simply continue comparing until `left` becomes equal to `loc`. Once `left = loc`, it means the pivot has been placed in its correct position.
 - (b) However, if at any point, we have `a[loc] < a[left]`, then interchange the two values and jump to Step 2.
 - (c) Set `loc = left`.

Consider the following unsorted list of elements...

List

5	3	8	1	4	6	2	7
---	---	---	---	---	---	---	---

Define pivot, left & right. Set pivot = 0, left = 1 & right = 7. Here '7' indicates 'size-1'.

List

	left						right
5	3	8	1	4	6	2	7
pivot							

Compare List[left] with List[pivot]. If **List[left]** is greater than **List[pivot]** then stop left otherwise move left to the next.

Compare List[right] with List[pivot]. If List[right] is smaller than List[pivot] then stop right otherwise move right to the previous.

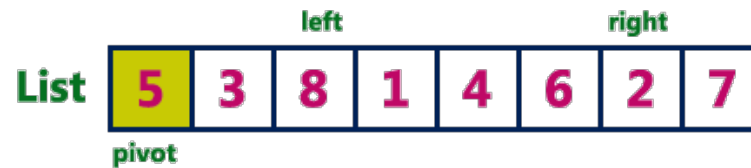
Repeat the same until **left >= right**.

If both left & right are stoped but $\text{left} < \text{right}$ then swap List[left] with List[right] and countinue the process.

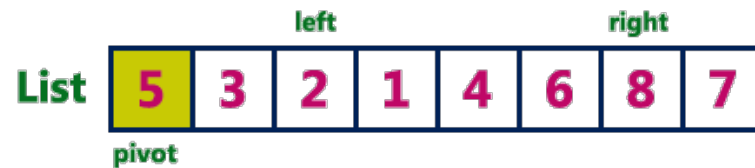
If $\text{left} >= \text{right}$ then swap List[pivot] with List[right].



Compare $\text{List}[\text{left}] < \text{List}[\text{pivot}]$ as it is true increment left by one and repeat the same, left will stop at 8.
Compare $\text{List}[\text{right}] > \text{List}[\text{pivot}]$ as it is true decrement right by one and repeat the same, right will stop at 2.

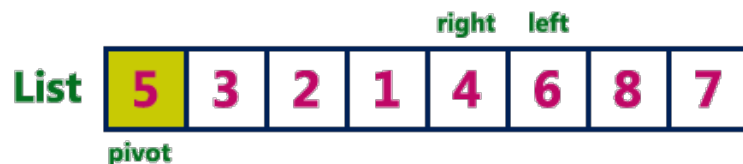


Here left & right both are stopped and left is not greater than right so we need to swap $\text{List}[\text{left}]$ and $\text{List}[\text{right}]$



Compare `List[right]>List[pivot]` as it is true decrement right by one and repeat the same, right will stop at 4.

Compare `List[right]>List[pivot]` as it is true decrement right by one and repeat the same, right will stop at 4.



Here left & right both are stopped and left is greater than right so we need to swap List[pivot] and List[right]



Here we can observe that all the numbers to the left side of 5 are smaller and right side are greater. That means 5 is placed in its correct position.

Repeat the same process on the left sublist and right sublist to the number 5.



In the left sublist as there are no smaller number than the pivot left will keep on moving to the next and stops at last number. As the List[right] is smaller, right stops at same position. Now left and right both are equal so we swap pivot with right.





In the right sublist left is greater than the pivot, left will stop at same position.
As the List[right] is greater than List[pivot], right moves towards left and stops at pivot number position.
Now left > right so we swap pivot with right. (6 is swap by itself).



Repeat the same recursively on both left and right sublists until all the numbers are sorted.
The final sorted list will be as follows...



Insertion Sort – Exercise

Sort following array using Quick Sort.

1. {25,15,30,19,99,45,26}
2. {44,77,22,88,33,66,11,55}

Complexity of Quick Sort

- In the average case, the running time of quick sort can be given as $O(n \log n)$. The partitioning of the array which simply loops over the elements of the array once uses $O(n)$ time.
- In the best case, every time we partition the array, we divide the list into two nearly equal pieces. That is, the recursive call processes the sub-array of half the size. At the most, only $\log n$ nested calls can be made before we reach a sub-array of size 1. It means the depth of the call tree is $O(\log n)$. And because at each level, there can only be $O(n)$, the resultant time is given as $O(n \log n)$ time.
- Its worst-case efficiency is given as $O(n^2)$.

Merge Sort

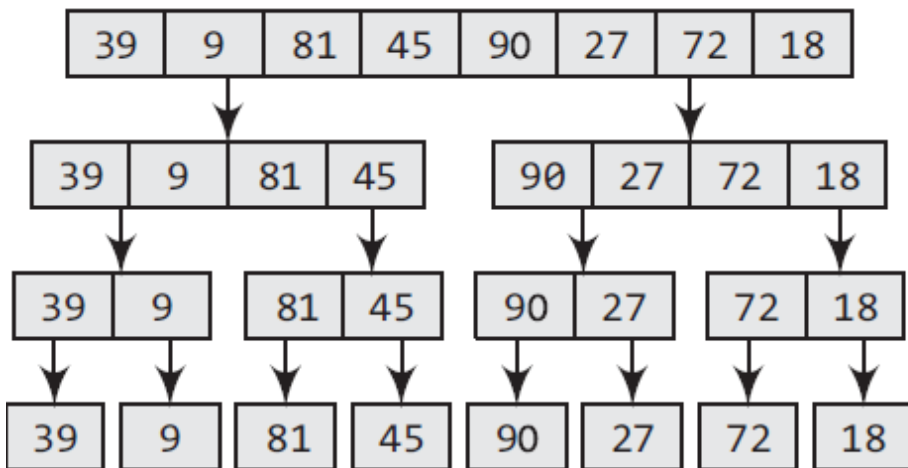
- Merge sort is a sorting algorithm that uses the divide, conquer, and combine algorithmic paradigm.
- **Divide** means partitioning the n -element array to be sorted into two sub-arrays of $n/2$ elements.
 - If A is an array containing zero or one element, then it is already sorted. However, if there are more elements in the array, divide A into two sub-arrays, A_1 and A_2 , each containing about half of the elements of A .
- **Conquer** means sorting the two sub-arrays recursively using merge sort.
- **Combine** means merging the two sorted sub-arrays of size $n/2$ to produce the sorted array of n elements.

- 
- Merge sort algorithm focuses on two main concepts to improve its performance (running time):
 - A smaller list takes fewer steps and thus less time to sort than a large list.
 - As number of steps is relatively less, thus less time is needed to create a sorted list from two sorted lists rather than creating it using two unsorted lists.

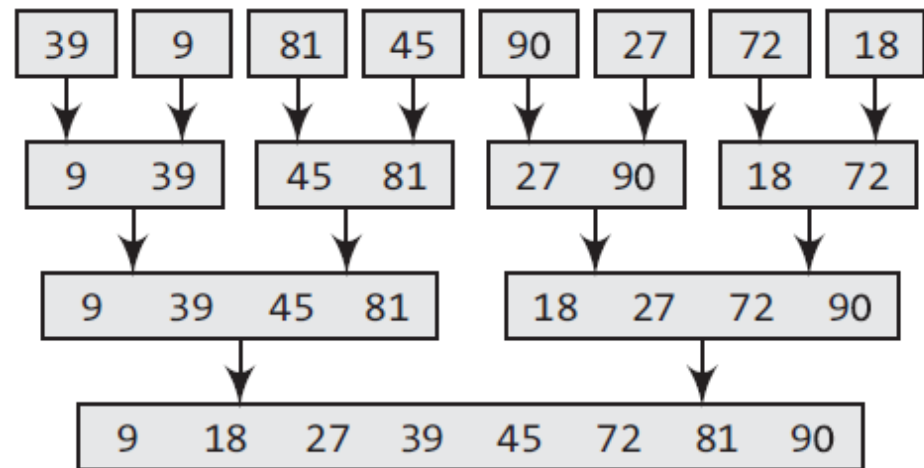
- 
- The basic steps of a merge sort algorithm are as follows:
- If the array is of length 0 or 1, then it is already sorted.
 - Otherwise, divide the unsorted array into two sub-arrays of about half the size.
 - Use merge sort algorithm recursively to sort each sub-array.
 - Merge the two sub-arrays to form a single sorted list.



39	9	81	45	90	27	72	18
----	---	----	----	----	----	----	----



(Divide and Conquer the array)



(Combine the elements to form a sorted array)

MERGE_SORT(ARR, BEG, END)

Step 1: IF BEG < END

 SET MID = (BEG + END)/2

 CALL MERGE_SORT (ARR, BEG, MID)

 CALL MERGE_SORT (ARR, MID + 1, END)

 MERGE (ARR, BEG, MID, END)

 [END OF IF]

Step 2: END

- The running time of merge sort in the average case and the worst case can be given as $O(n \log n)$.

Radix Sort

- Radix sort is one of the sorting algorithms used to sort a list of integer numbers in order.
- In radix sort algorithm, a list of integer numbers will be sorted based on the digits of individual numbers.
- Sorting is performed from the least significant digit to the most significant digit.

- 
- Radix sort algorithm requires the number of passes which are equal to the number of digits present in the largest number among the list of numbers.
 - For example, if the largest number is a 3-digit number, then that list is sorted with 3 passes.



➤ The Radix sort algorithm is performed using the following steps

- **Step 1** - Define 10 queues each representing a bucket for each digit from 0 to 9.
- **Step 2** - Consider the least significant digit of each number in the list which is to be sorted.
- **Step 3** - Insert each number into their respective queue based on the least significant digit.
- **Step 4** - Group all the numbers from queue 0 to queue 9 in the order they have inserted into their respective queues.
- **Step 5** - Repeat from step 3 based on the next least significant digit.
- **Step 6** - Repeat from step 2 until all the numbers are grouped based on the most significant digit.



Consider the following list of unsorted integer numbers

82, 901, 100, 12, 150, 77, 55 & 23

Step 1 - Define 10 queues each represents a bucket for digits from 0 to 9.



Step 2 - Insert all the numbers of the list into respective queue based on the Least significant digit (once placed digit) of every number.

82, 901, 1000, 12, 1500, 770, 550 & 230



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 150, 901, 82, 12, 23, 55 & 77

Step 3 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Tens placed digit) of every number.

100, 150, 901, 82, 12, 23, 55 & 77



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 901, 12, 23, 150, 55, 77 & 82

Step 4 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Hundred's place digit) of every number.

100, 901, 12, 23, 150, 55, 77 & 82



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

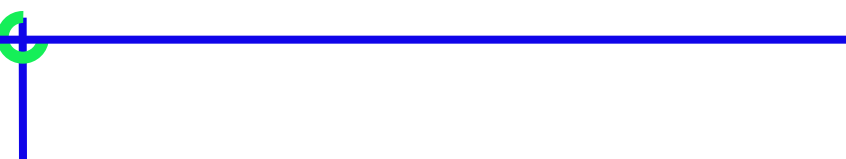
12, 23, 55, 77, 82, 100, 150, 901

List got sorted in the increasing order.

Complexity of the Radix Sort Algorithm

- To sort an unsorted list with 'n' number of elements, Radix sort algorithm needs the following complexities...
- Worst Case : $O(n)$
- Best Case : $O(n)$
- Average Case : $O(n)$

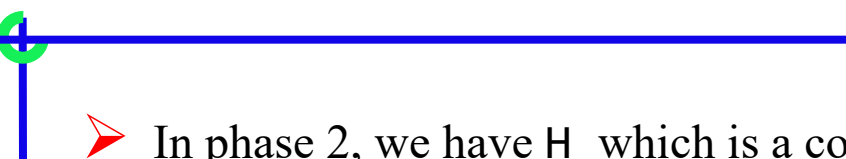
Heap Sort

- 
- Given an array ARR with n elements, the heap sort algorithm can be used to sort ARR in two phases:
- In phase 1, build a heap H using the elements of ARR .
 - In phase 2, repeatedly delete the root element of the heap formed in phase 1.

Complexity of Heap Sort

- Heap sort uses two heap operations: insertion and root deletion.
- Each element extracted from the root is placed in the last empty location of the array.

- 
- In phase 1, when we build a heap, the number of comparisons to find the right location of the new element in H cannot exceed the depth of H . Since H is a complete tree, its depth cannot exceed m , where m is the number of elements in heap H .
 - Thus, the total number of comparisons $g(n)$ to insert n elements of ARR in H is bounded as: $g(n) \leq n \log n$
 - Hence, the running time of the first phase of the heap sort algorithm is $O(n \log n)$.

- 
- In phase 2, we have H which is a complete tree with m elements having left and right sub-trees as heaps. Assuming L to be the root of the tree, *reheaping* the tree would need 4 comparisons to move L one step down the tree H .
 - Since the depth of H cannot exceed $O(\log m)$, reheaping the tree will require a maximum of $4 \log m$ comparisons to find the right location of L in H .
 - Since n elements will be deleted from heap H , reheaping will be done n times. Therefore, the number of comparisons to delete n elements is bounded as: $h(n) \leq 4n \log n$
 - Hence, the running time of the second phase of the heap sort algorithm is $O(n \log n)$.

- 
- Each phase requires time proportional to $O(n \log n)$. Therefore, the running time to sort an array of n elements in the worst case is proportional to $O(n \log n)$.



Searching Algorithms

What is Search?



➤ Search is a process of finding a value in a list of values. In other words, searching is the process of locating given value position in a list of values.

- Two types of searching
- Linear Search Algorithm
 - Binary Search Algorithm

Linear Search Algorithm

- This search process starts comparing of search element with the first element in the list.
 - If both are matching then results with element found otherwise search element is compared with next element in the list.
 - If both are matched, then the result is "element found". Otherwise, repeat the same with the next element in the list until search element is compared with last element in the list, if that last element also doesn't match, then the result is "Element not found in the list".
- That means, the search element is compared with element by element in the list.

Algorithm

- Linear search is implemented using following steps...
- **Step 1:** Read the search element from the user
- **Step 2:** Compare, the search element with the first element in the list.
- **Step 3:** If both are matching, then display "Given element found!!!" and terminate the function
- **Step 4:** If both are not matching, then compare search element with the next element in the list.
- **Step 5:** Repeat steps 3 and 4 until the search element is compared with the last element in the list.
- **Step 6:** If the last element in the list is also doesn't match, then display "Element not found!!!" and terminate the function.

Algorithm

Linear Search (Array A, Value x)

- Step 1: Set i to 1
- Step 2: if $i > n$ then go to step 7
- Step 3: if $A[i] = x$ then go to step 6
- Step 4: Set i to $i + 1$
- Step 5: Go to Step 2
- Step 6: Print Element x Found at index i and go to step 8
- Step 7: Print element not found
- Step 8: Exit

Example

Consider the following list of element and search element...

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

search element **12**

Step 1:

search element (12) is compared with first element (65)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 2:

search element (12) is compared with next element (20)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 3:

search element (12) is compared with next element (10)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99
			12					

Both are not matching. So move to next element

Step 4:

search element (12) is compared with next element (55)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99
				12				

Both are not matching. So move to next element

Step 5:

search element (12) is compared with next element (32)

	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are not matching. So move to next element

Step 6:

search element (12) is compared with next element (12)

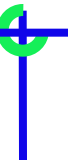
	0	1	2	3	4	5	6	7
list	65	20	10	55	32	12	50	99

12

Both are matching. So we stop comparing and display element found at index 5.

Binary Search Algorithm

- The binary search algorithm can be used with only sorted list of element. That means, binary search can be used only with list of element which are already arranged in a order.
- The binary search can not be used for list of element which are in random order.

- 
- This search process starts comparing of the search element with the middle element in the list. If both are matched, then the result is "element found".
 - Otherwise, we check whether the search element is smaller or larger than the middle element in the list.
 - If the search element is smaller, then we repeat the same process for left sublist of the middle element.
 - If the search element is larger, then we repeat the same process for right sublist of the middle element.
 - We repeat this process until we find the search element in the list or until we left with a sublist of only one element. And if that element also doesn't match with the search element, then the result is "Element not found in the list".

Algorithm

Binary search is implemented using following steps...

- **Step 1:** Read the search element from the user
- **Step 2:** Find the middle element in the sorted list
- **Step 3:** Compare, the search element with the middle element in the sorted list.
- **Step 4:** If both are matching, then display "Given element found!!!" and terminate the function
- **Step 5:** If both are not matching, then check whether the search element is smaller or larger than middle element.
- **Step 6:** If the search element is smaller than middle element, then repeat steps 2, 3, 4 and 5 for the left sublist of the middle element.
- **Step 7:** If the search element is larger than middle element, then repeat steps 2, 3, 4 and 5 for the right sublist of the middle element.
- **Step 8:** Repeat the same process until we find the search element in the list or until sublist contains only one element.
- **Step 9:** If that element also doesn't match with the search element, then display "Element not found in the list!!!" and terminate the function.

Algorithm

Procedure binary_search

$A \leftarrow$ sorted array

$n \leftarrow$ size of array

$x \leftarrow$ value to be searched

Set lowerBound = 1 Set

upperBound = n

while x not found

if upperBound < lowerBound

EXIT: x does not exists.

set midPoint = lowerBound + (
upperBound - lowerBound) / 2

if $A[\text{midPoint}] < x$

set lowerBound = midPoint + 1

if $A[\text{midPoint}] > x$

set upperBound = midPoint - 1

if $A[\text{midPoint}] = x$

EXIT: x found at location midPoint

end while

end procedure

Example

Consider the following list of element and search element...

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

search element **12**

Step 1:

search element (12) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
					12				

Both are not matching. And 12 is smaller than 50. So we search only in the left sublist (i.e. 10, 12, 20 & 32).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (12) is compared with middle element (12)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
		12							

Both are matching. So the result is "Element found at index 1"

search element **80**

Step 1:

search element (80) is compared with middle element (50)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
					80				

Both are not matching. And 80 is larger than 50. So we search only in the right sublist (i.e. 55, 65, 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 2:

search element (80) is compared with middle element (65)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
							80		

Both are not matching. And 80 is larger than 65. So we search only in the right sublist (i.e. 80 & 99).

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99

Step 3:

search element (80) is compared with middle element (80)

	0	1	2	3	4	5	6	7	8
list	10	12	20	32	50	55	65	80	99
								80	

Both are not matching. So the result is "Element found at index 7"

Topics remain

- 
- Red Black Tree
 - Threaded Binary Tree
 - Shortest Path Algorithms
 - Hashing
 - File Organization